

DOCUMENTACIÓN COMPLETA: PROYECTO CORE TATA

Sistema Bancario Core - Arquitectura y Funcionamiento

Versión: 1.0
Fecha de Documento: 19 de Mayo de 2026
Proyecto: CORE TATA - Sistema Bancario Integrado
Stack: COBOL / MySQL 8.0 / GnuCOBOL 4.x

TABLA DE CONTENIDOS

- 1. [Visión General](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Componentes Principales](#)
- 4. [Flujos de Datos](#)
- 5. [Estructura de Base de Datos](#)
- 6. [Procesos Batch](#)
- 7. [Guía de Operación](#)
- 8. [Dependencias Técnicas](#)

VISIÓN GENERAL

1.1 ¿Qué es CORE TATA?

CORE TATA es un sistema integral de procesamiento bancario implementado en **COBOL** con arquitectura moderna de **procesamiento en paralelo** (Trickle Feed). El sistema maneja operaciones críticas de un banco incluyendo:

- ✓ Gestión de clientes y cuentas corrientes
- ✓ Transacciones en línea (débitos, créditos, consultas)
- ✓ Tarjetas de crédito
- ✓ Hipotecas y préstamos
- ✓ **Procesamiento asíncrono de lotes masivos** (25k+ transacciones)
- ✓ Cierre mensual con consolidación automática
- ✓ Reportería gerencial

1.2 Tecnologías Principales

Componente	Tecnología	Versión
Lenguaje Principal	COBOL	GnuCOBOL 4.x (2024.04.30)
Pre-compilador SQL	esqlOC	v3 (OpenCOBOL SQL)
Base de Datos	MySQL / MariaDB	8.0+
SO Destino	Windows	XP SP3+

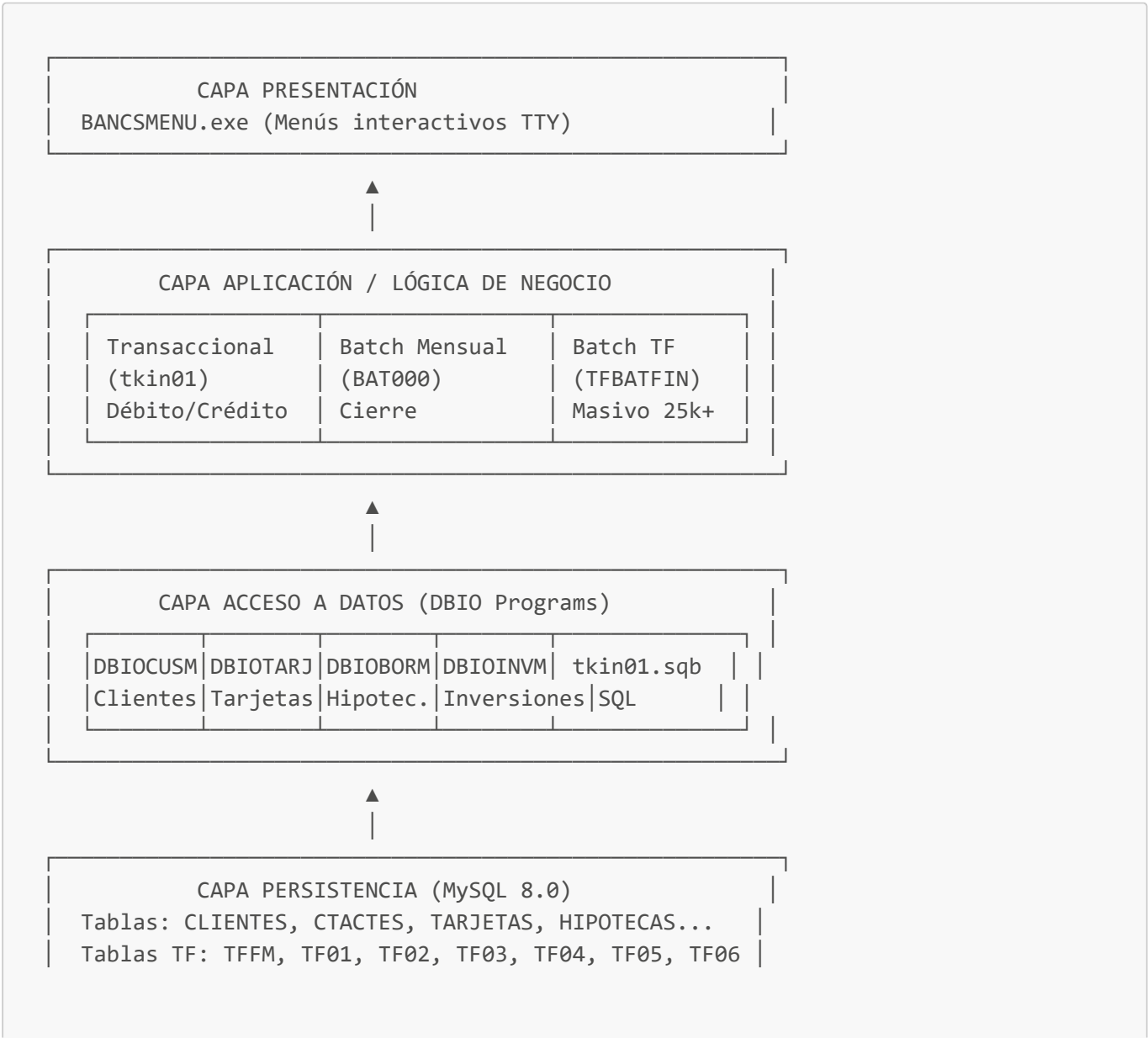
Componente	Tecnología	Versión
Interface	Línea de comandos	Menús interactivos TTY

1.3 Estadísticas de Proyecto

Líneas de código COBOL:	~15,000 LOC
Programas principales:	18
Archivos SQL embebido:	12 (.sqb)
Copybooks/Headers:	7 (LKTF, PATHS, LKCIF, etc.)
Scripts batch:	3 (compilar, ejecutar, monitor)
Tablas de BD:	14+
Base de datos:	proyecto_cobol (MySQL 8.0)

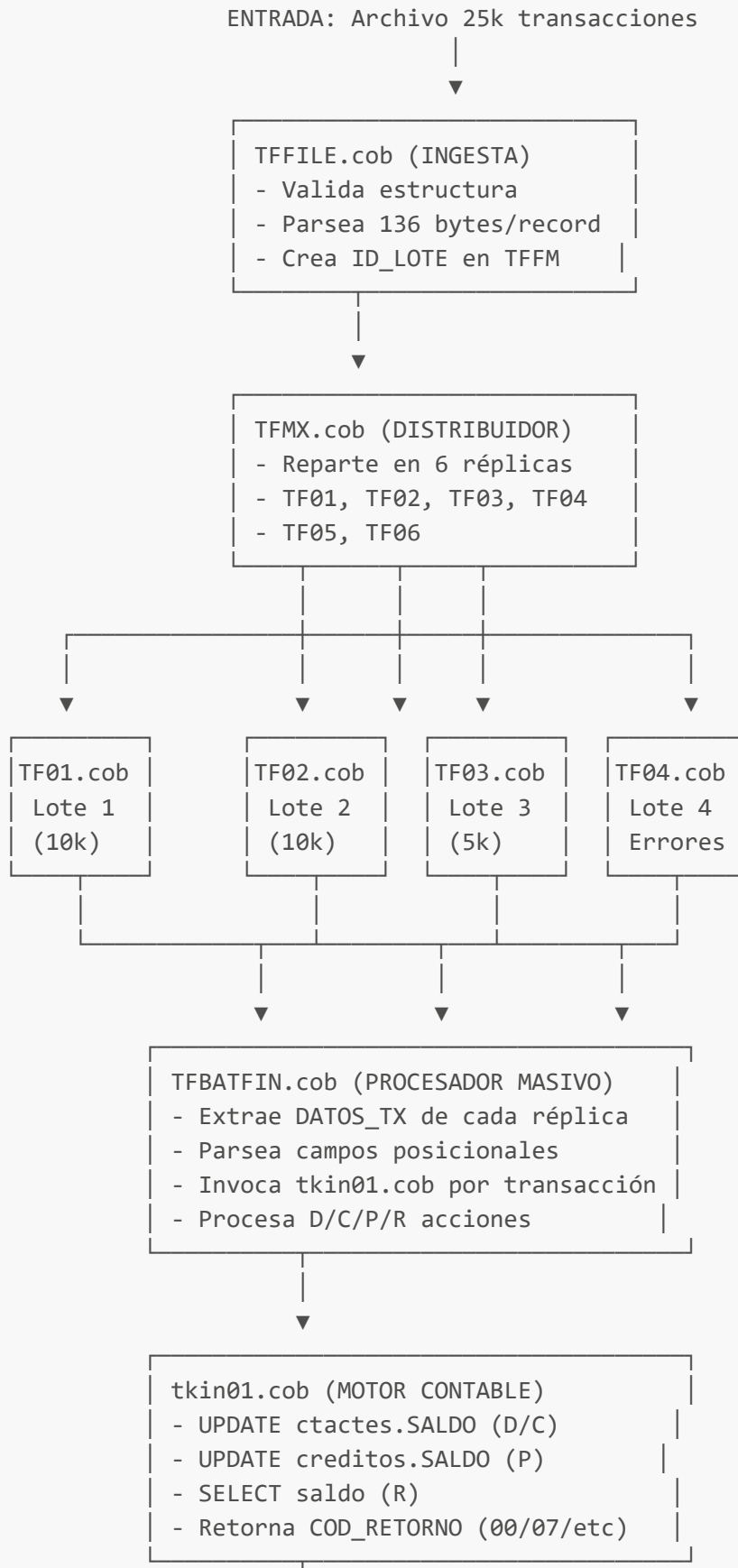
ARQUITECTURA DEL SISTEMA

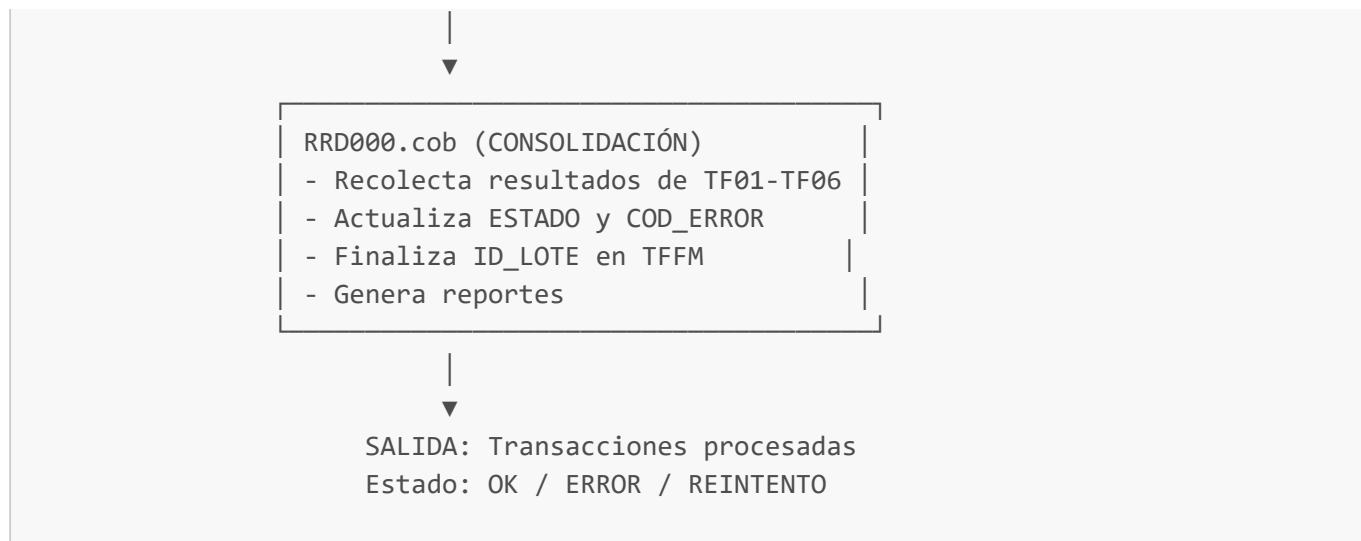
2.1 Visión de Capas



Auditoría: AUDIT_MAESTRA, AUDIT_TARJETAS, etc.

2.2 Procesamiento Trickle Feed (Arquitectura Paralela)





COMPONENTES PRINCIPALES

3.1 Programas Principales (src/mainline/)

3.1.1 BANCSMENU.cob - Punto de Entrada Principal

Responsabilidad: Interface de usuario principal - menú interactivo TTY

Funcionalidad:

```
BANCSMENU.exe (Main Entry Point)
├─ Opción 1: Gestión de Clientes
│   └─ CALL DBIOCUSM (INSERT/UPDATE/SELECT)
├─ Opción 2: Gestión de Cuentas Corrientes
│   └─ CALL tkin01 (Débitos, créditos, consultas)
├─ Opción 3: Gestión de Tarjetas de Crédito
│   └─ CALL DBIOTARJ (Tarjetas CRUD)
├─ Opción 4: Gestión de Hipotecas/Préstamos
│   └─ CALL DBIOBORM (Hipotecas CRUD)
├─ Opción 5: Cierre Mensual (Batch)
│   └─ CALL BAT000 (Consolidación mensual)
├─ Opción 6: Reportería Gerencial
│   └─ CALL RP0000 (Reportes)
└─ Opción 7: Procesamiento Trickle Feed
    └─ CALL TFDRMAIN (Batch masivo asincrónico)
```

Variables globales (LKCIF.CPY):

- **WS-USUARIO:** Usuario actual

- **WS-TERMINAL**: Terminal de acceso
- **WS-SESION-ACTIVA**: Estado de sesión

3.1.2 tkin01.cob/sqb - Motor Contable (CRÍTICO)

Responsabilidad: Operaciones de débito, crédito y consulta de saldos

Operaciones soportadas:

ACCION	Operación	SQL	COD_RETORNO
D	Débito	UPDATE ctactes SET SALDO=SALDO-monto	00 = OK, 07 = Saldo insuficiente
C	Crédito	UPDATE ctactes SET SALDO=SALDO+monto	00 = OK
P	Pago	UPDATE creditos SET SALDO_PENDIENTE=...	00 = OK, 08 = Crédito no existe
R	Consulta	SELECT SALDO FROM ctactes	00 = OK, 01 = Cuenta no existe

Interfaz (LKTF.CPY):

```

01  WS-TKIN-INTERFAZ.
    05  ACCION          PIC X(1).    *> D/C/P/R
    05  ID-CUENTA       PIC 9(10).
    05  IMPORTE         PIC 9(15).    *> En centavos
    05  NUM-CREDITO     PIC 9(10).
    05  COD-RETORNO     PIC 9(2).     *> Código error
    05  SALDO-ACTUAL    PIC 9(15).    *> Output

```

Lógica:

```

IF ACCION = 'D' (Débito)
  EXEC SQL
    UPDATE ctactes
    SET SALDO_ACTUAL = SALDO_ACTUAL - :IMPORTE
    WHERE ID_CLIENTE = :ID-CUENTA
    AND SALDO_ACTUAL >= :IMPORTE
  END-EXEC
  IF NOT FOUND → COD-RETORNO = 07

ELSE IF ACCION = 'C' (Crédito)
  EXEC SQL
    UPDATE ctactes
    SET SALDO_ACTUAL = SALDO_ACTUAL + :IMPORTE
    WHERE ID_CLIENTE = :ID-CUENTA
  END-EXEC
  IF ROWS-AFFECTED = 0 → COD-RETORNO = 01

```

3.1.3 BAT000.cob/sqb - Cierre Mensual

Responsabilidad: Consolidación de saldos, mora, y finalización de mes

Procesos ejecutados:

1. Consolidación de Clientes

```
DECLARE c_clientes CURSOR FOR
SELECT ID_CLIENTE, SALDO_ACTUAL, ESTADO
FROM CLIENTES
WHERE ESTADO != 'INACTIVO'
```

- Recorre cada cliente
- Consolida SALDO_ACTUAL en CTACTES
- Registra en AUDIT_MAESTRA

2. Procesamiento de Tarjetas

- Identifica tarjetas vencidas
- Descuenta saldo utilizado
- Genera aviso de vencimiento
- Registra en AUDIT_TARJETAS

3. Procesamiento de Hipotecas

- Calcula cuota mensual a vencer
- Descuenta automáticamente si hay fondos
- Identifica mora (30+ días atraso)
- Aplica interés moratorio
- Registra en AUDIT_HIPOTECAS

4. Consolidación Final

```
INSERT INTO AUDIT_MAESTRA (
    FECHA_PROCESO, TIPO_CONSOLIDACION,
    SALDO_INICIAL, SALDO_FINAL,
    TRANSACCIONES_PROCESADAS
) VALUES (...)
```

3.1.4 TFFILE.cob/sqb - Ingesta y Validación

Responsabilidad: Lectura de archivo entrada, validación estructura

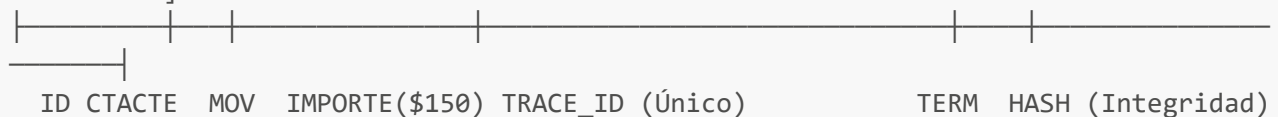
Procedimiento:

LECTURA DEL ARCHIVO

- └ Abre archivo en BATCH-INPUT/
- └ Lee línea a línea (136 bytes)
- └ PARSEO POSICIONAL (136 bytes exactos)
 - └ Bytes 1-10: ID_CUENTA (PIC 9(10))
 - └ Bytes 11-13: COD_MOV (002=DEP, 003=RET)
 - └ Bytes 14-28: IMPORTE (PIC 9(15))
 - └ Bytes 29-68: TRACE_ID (PIC X(40))
 - └ Bytes 69-72: TERMINAL (PIC X(4))
 - └ Bytes 73-136: HASH_SEG (SHA-256, PIC X(64))
- └ VALIDACIÓN
 - └ Verifica ID_CUENTA existe en BD
 - └ Verifica COD_MOV en rango válido (002, 003)
 - └ Verifica IMPORTE > 0
 - └ Valida HASH_SEG (integridad)
 - └ Si ERROR → Registra en TF06 (Errores)
- └ QUIEBRE EN LOTES
 - └ Máximo 10,000 registros por réplica
 - └ Quiebre automático: 10k + 10k + 5k para 25k registros
- └ REGISTRO EN TFFM
 - └ INSERT ID_LOTE (auto-increment)
 - └ NOMBRE_ARCHIVO
 - └ FASE = 00 (Ingresado)
 - └ TIPO_PROG = "EPG" (Depósito/Pago)
 - └ TOTAL_REGISTROS = contador

Formato de entrada (136 bytes posicionales):

```
0000000001|002|000000000015000|TR-STRESS-20260517-0000000001|ATM1|[SHA256: 64
caracteres]
```

**3.1.5 TFMX.cob/sqb - Distribuidor de Réplicas**

Responsabilidad: Distribuir registros validados en 6 réplicas (TF01-TF06)

Lógica:

```
INPUT: Registros validados de TFFILE
|
FOR EACH validado_registro
```

```

├─ Determina ID_LOTE (de TFFM)
├─ Determina réplica destino (REPLICA_NO = 1-6)
│   └─ Si lote 1: REPLICA_NO = 1
│   └─ Si lote 2: REPLICA_NO = 2
│   └─ Si lote 3: REPLICA_NO = 3
│   └─ Si ERROR: REPLICA_NO = 6 (Tabla de errores)
└─ INSERT INTO TF[REPLICA_NO]
    ├─ ID_LOTE
    ├─ ESTADO = 1 (Ingresado)
    ├─ DATOS_TX = registro raw 136 bytes
    ├─ CUENTA = ID_CUENTA (parsed)
    ├─ MONTO = IMPORTE (parsed)
    ├─ TRACE_ID = TRACE_ID (parsed)
    ├─ TYPE_UPDATE = COD_MOV (D/C/P/R)
    └─ REPLICA_NO = 1-6

```

OUTPUT: 6 réplicas pobladas (TF01-TF06)

3.1.6 TFBATFIN.cob/sqb - Procesador Masivo (Paralelizado)

Responsabilidad: Procesar transacciones de 6 réplicas en paralelo

Pseudocódigo:

```

PROCEDURE TFBATFIN.
  FOR EACH ID_LOTE (en TFFM)
    FOR REPLICA = 1 TO 6
      FOR EACH registro IN TF[REPLICA]
        WHERE ESTADO = 1 (Ingresado)

        *> EXTRAE CAMPOS
        MOVE DATOS_TX a WS-RAW-REGISTRO
        MOVE BYTES 1-10 a WS-ID-CUENTA
        MOVE BYTES 11-13 a WS-COD-MOV
        MOVE BYTES 14-28 a WS-IMPORTE

        *> MAPEA ACCION
        EVALUATE WS-COD-MOV
          WHEN 002 MOVE 'C' TO WS-ACCION *> Crédito
          WHEN 003 MOVE 'D' TO WS-ACCION *> Débito
          WHEN OTHER MOVE 'R' TO WS-ACCION *> Consulta
        END-EVALUATE

        *> LLAMA MOTOR CONTABLE
        CALL 'tkin01' USING WS-ACCION,
                           WS-ID-CUENTA,
                           WS-IMPORTE,
                           WS-COD-RETORNO,

```



```

                                WS-SALDO

                                *> REGISTRA RESULTADO
                                MOVE WS-COD-RETORNO TO COD_ERROR
                                IF WS-COD-RETORNO = 00
                                    UPDATE TF[REPLICA]
                                    SET ESTADO = 3 (Procesado OK)
                                ELSE
                                    UPDATE TF[REPLICA]
                                    SET ESTADO = 4 (Error)
                                    SET COD_ERROR = WS-COD-RETORNO

                                END-FOR
                                END-FOR
                                END-FOR
                                END PROCEDURE.

```

Paralelización: Cada réplica (TF01-TF06) se procesa independientemente, permitiendo:

- Velocidad: ~4k transacciones/minuto por réplica
- Capacidad: 24k transacciones/minuto en 6 réplicas (25k en 1.04 min)

3.1.7 RRD000.cob - Consolidación y Reportes

Responsabilidad: Recolectar resultados de 6 réplicas, generar reportes finales

Proceso:

```

FOR EACH ID_LOTE IN TFFM
    TOTAL_OK = 0
    TOTAL_ERROR = 0
    TOTAL_REINTENTO = 0

    FOR EACH registro IN TF01-TF06
        EVALUATE ESTADO
            WHEN 3 ADD 1 TO TOTAL_OK
            WHEN 4 ADD 1 TO TOTAL_ERROR
                ADD 1 TO TOTAL_REINTENTO (luego)
            WHEN OTHER ADD 0
        END-EVALUATE

    *> GENERA REPORTE
    INSERT INTO TRICKLE_FEED_REPORT
        ID_LOTE, TOTAL_PROCESADOS, OK, ERROR,
        FECHA_REPORT, TIEMPO_PROCESAMIENTO

    *> FINALIZA LOTE EN TFFM
    UPDATE TFFM
    SET FASE = 40 (Finalizado)

```

```
SET STATUS = 'EXITOSO'  
WHERE ID_LOTE = :ID-LOTE
```

3.1.8 DBIOCUSM.cob/sqb - Gestión de Clientes

Operaciones:

- INSERT nuevo cliente
- UPDATE datos cliente
- SELECT cliente por documento
- UPDATE estado cliente (ACTIVO/INACTIVO)

SQL Embebido:

```
EXEC SQL  
    SELECT ID_CLIENTE, TIPO_DOC, DOC_CLIENTE, NOMBRE  
    FROM CLIENTES  
    WHERE DOC_CLIENTE = :P-DOC-CLIENTE  
    AND TIPO_DOC = :P-TIPO-DOC  
END-EXEC  
  
EXEC SQL  
    INSERT INTO CLIENTES  
    (TIPO_DOC, DOC_CLIENTE, NOMBRE, ESTADO, FECHA_ALTA)  
    VALUES (:P-TIPO-DOC, :P-DOC-CLIENTE, :P-NOMBRE, 'ACTIVO', NOW())  
END-EXEC  
  
EXEC SQL  
    UPDATE CLIENTES  
    SET ESTADO = :P-ESTADO  
    WHERE ID_CLIENTE = :P-ID-CLIENTE  
END-EXEC
```

3.1.9 Otros Programas (Resumen)

Programa	Función
DBIOTARJ	CRUD de tarjetas de crédito
DBIOBORM	CRUD de hipotecas/préstamos
DBIOINVM	CRUD de inversiones
DBIOTRAN	Orquestador de transacciones
RP0000	Generador de reportes gerenciales
BNCR004	Consulta de fechas de proceso

Programa	Función
testconn	Validador de conexión BD
BR0000, CI0000, DF0000 , etc.	Módulos de soporte

3.2 Archivos SQL Embebido (sql/*.sqb)

Los archivos **.sqb** contienen SQL embebido en COBOL que se precompila con **esqlOC.exe**:

Archivo	Embebido en	Funcionalidad
tkin01.sqb	tkin01.cob	Motor contable (UPDATE/SELECT saldos)
BAT000.sqb	BAT000.cob	Cursorres cierre mensual
DBIOCUSM.sqb	DBIOCUSM.cob	Gestión clientes
DBIOBORM.sqb	DBIOBORM.cob	Gestión hipotecas
DBIOINVM.sqb	DBIOINVM.cob	Gestión inversiones
DBIOTARJ.sqb	DBIOTARJ.cob	Gestión tarjetas
TFFILE.sqb	TFFILE.cob	Lectura lotes TF
TFMX.sqb	TFMX.cob	Distribución a réplicas
TFBATFIN.sqb	TFBATFIN.cob	Procesamiento masivo

3.3 Copybooks (src/copies/)

Archivos compartidos de estructura de datos:

3.3.1 LKTF.CPY - Interfaz Trickle Feed

```

01  WS-TRICKLE-FEED-INTERFAZ.
    05  ID-LOTE                PIC 9(10).
    05  ACCION                 PIC X(1).      *> D/C/P/R
    05  ID-CUENTA              PIC 9(10).
    05  IMPORTE                PIC 9(15).
    05  NUM-CREDITO            PIC 9(10).
    05  COD-RETORNO            PIC 9(2).
    05  ESTADO-LOTE           PIC 9(1).      *> 1/2/3/4
    05  REPLICA-NO             PIC 9(1).      *> 1-6
    05  DATOS-TX               PIC X(136).    *> Raw
    05  TRACE-ID               PIC X(40).
```

3.3.2 LKCIF.CPY - Área de Sesión

```

01  WS-SESSION-USUARIO.
    05  USUARIO              PIC X(15).
    05  TERMINAL             PIC X(4).
    05  FECHA-SESSION        PIC 9(8).
    05  HORA-SESSION         PIC 9(6).
    05  SESSION-ACTIVA       PIC X(1).      *> S/N

```

3.3.3 PATHS.CPY - Rutas Centralizadas

```

01  WS-PATHS.
    05  PATH-INPUT           PIC X(100)
        VALUE ".\banco\spool\Interfaces\BATCH-INPUT\".
    05  PATH-UPLOAD          PIC X(100)
        VALUE ".\banco\spool\Interfaces\BATCH-UPLOAD-S\".
    05  PATH-TEMP            PIC X(100)
        VALUE ".\banco\spool\Interfaces\BATCH-UPLOADS-TEMP\".
    05  PATH-REPORTS         PIC X(100)
        VALUE ".\banco\spool\Interfaces\TRICKLE-FEED-REPORT\".
    05  PATH-DONE            PIC X(100)
        VALUE ".\banco\spool\Interfaces\BATCH-DONE\".

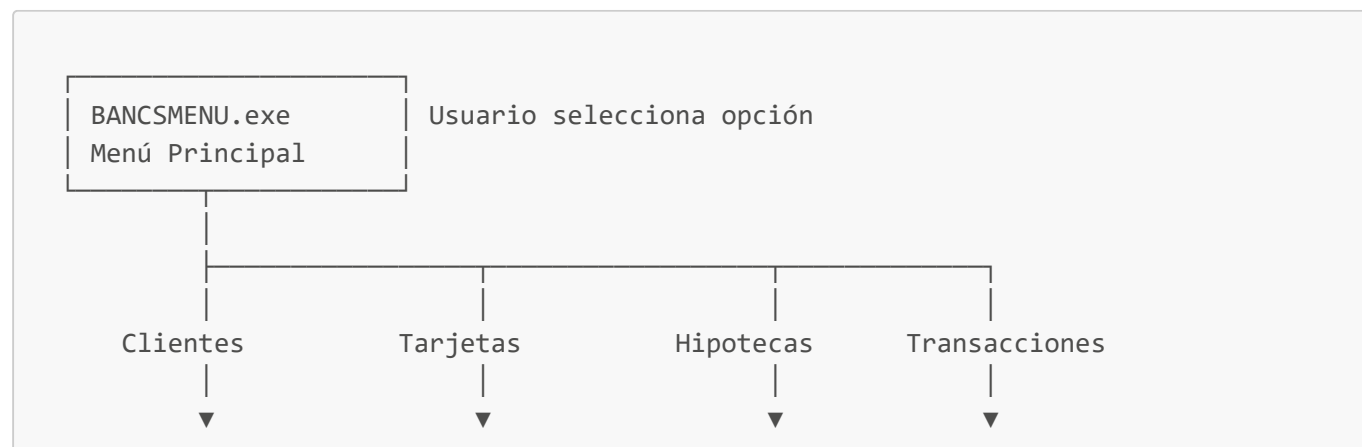
```

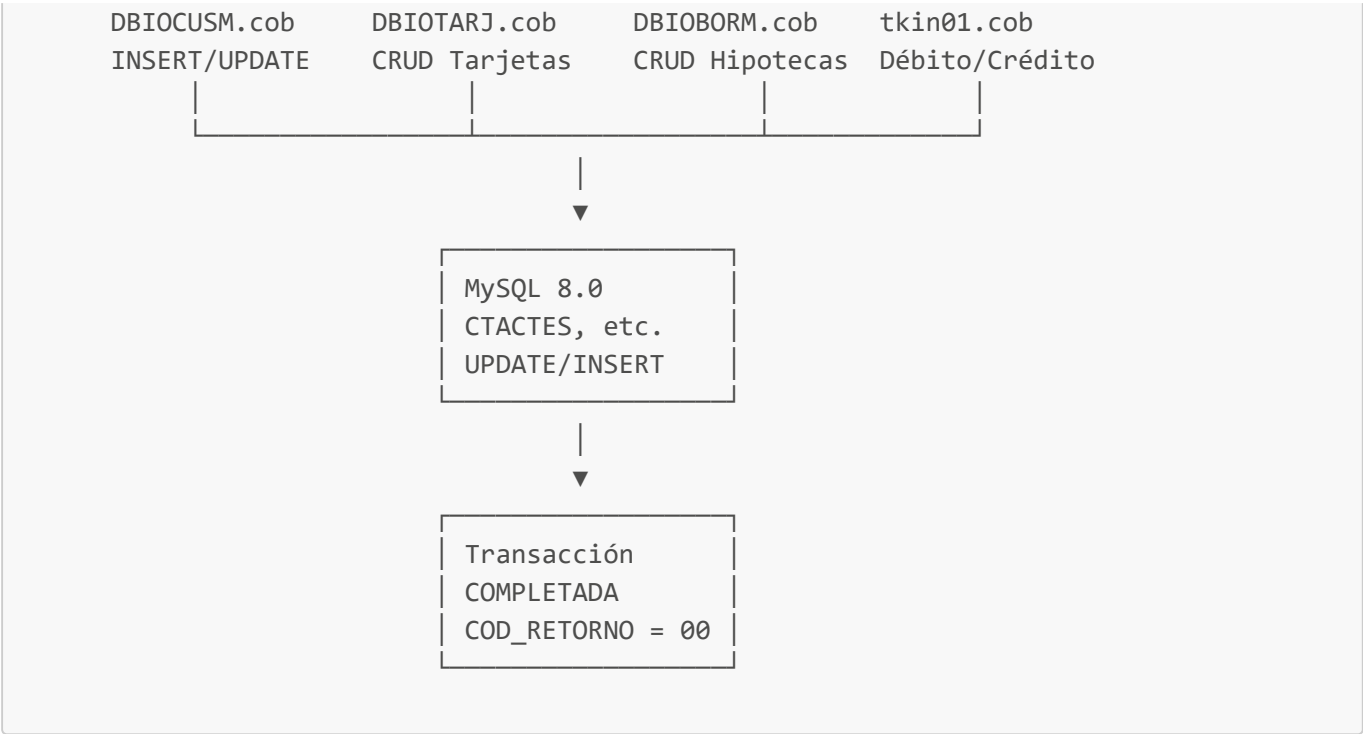
3.3.4 Copybooks de Registros

- **CUSMREC.CPY:** Estructura cliente
- **BORMREC.CPY:** Estructura hipoteca
- **TARJREC.CPY:** Estructura tarjeta
- **INVMREC.CPY:** Estructura inversión
- **SECREC.CPY:** Estructura seguridad
- **MOVREC.CPY:** Estructura movimiento
- **TIPOSCOM.cpy:** Constantes/tipos

FLUJOS DE DATOS

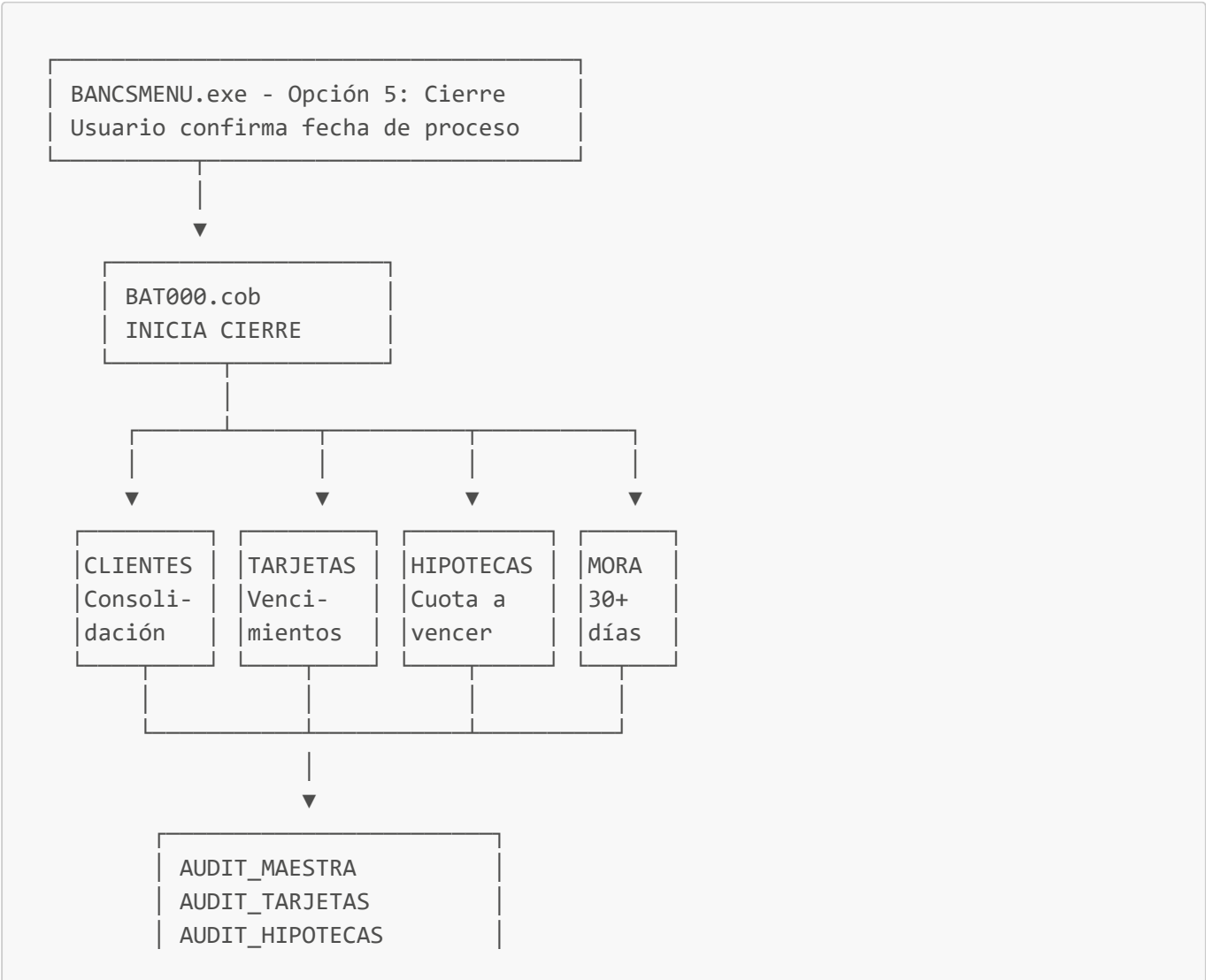
4.1 Flujo 1: Transaccional Interactivo (Online)

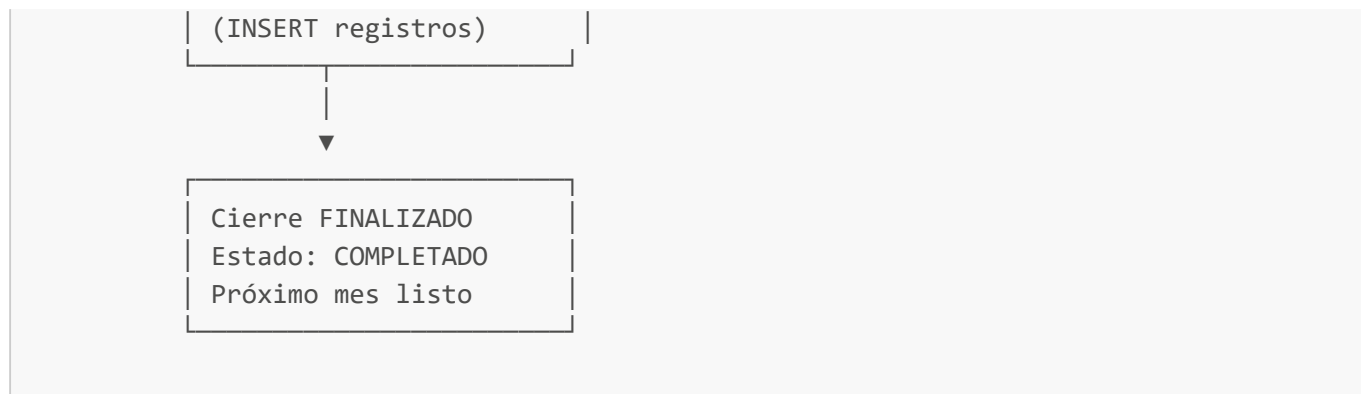




Tiempo de respuesta: < 100 ms por operación

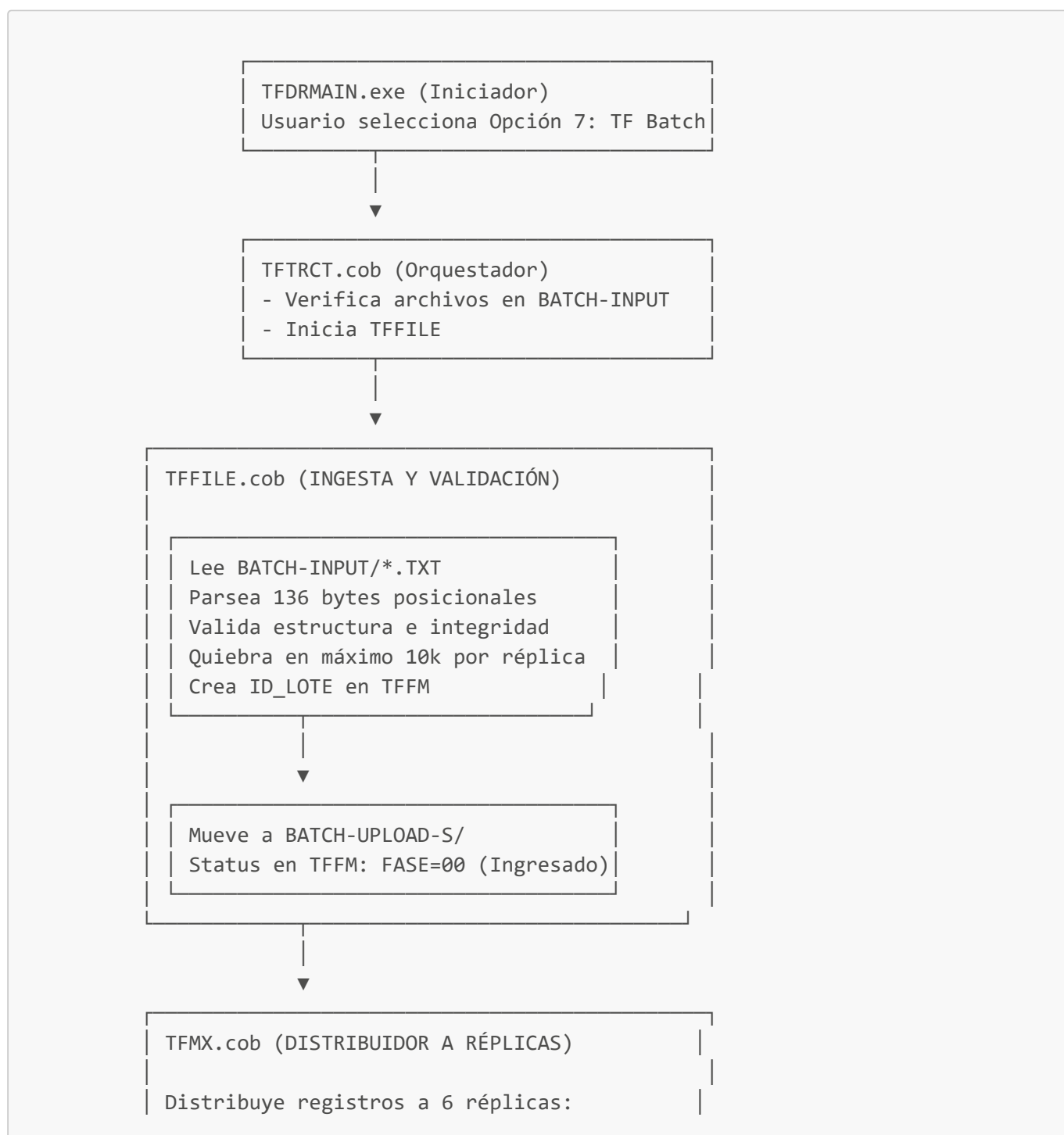
4.2 Flujo 2: Batch Monthly (Cierre de Mes)





Duración: ~30 minutos para 100k clientes

4.3 Flujo 3: Trickle Feed Masivo (Procesamiento en Paralelo)



```
└─ INSERT TF01 (Lote 1: 10k)
└─ INSERT TF02 (Lote 2: 10k)
└─ INSERT TF03 (Lote 3: 5k)
└─ INSERT TF04 (Lote 4+: overflow)
└─ INSERT TF05 (Errors/Reintento)
└─ INSERT TF06 (Failed)
```

Status en TFFM: FASE=05 (Distribuido)

▼ (PARALELIZACIÓN)

TFBATFIN.cob x 6 réplicas (PROCESADORES)

Réplica 1 (TF01)	Réplica 2 (TF02)	Réplica 3 (TF03)
10k trx	10k trx	5k trx
¡EN PARALELO!	¡EN PARALELO!	¡EN PARALELO!

Para cada registro:

1. Extrae DATOS_TX (136 bytes)
2. Parsea campos posicionales
3. CALL tkIn01 (Motor contable)
 - ACCION=D → UPDATE SALDO-
 - ACCION=C → UPDATE SALDO+
4. Registra COD_RETORNO (00/07/etc)
5. UPDATE TF[X] ESTADO=(3=OK/4=ERROR)

Status en TFFM: FASE=20-30 (Procesando)

▼

RRD000.cob (CONSOLIDACIÓN DE RESULTADOS)

Para cada ID_LOTE:

1. Itera TF01-TF06
2. Cuenta: OK, ERROR, REINTENTO
3. Genera REPORTE (TRICKLE_FEED_REPORT)
4. UPDATE TFFM: FASE=40 (Finalizado)
5. Mueve archivo a BATCH-DONE/

Status final: EXITOSO / FALLIDO

▼

SALIDA: 25k TRANSACCIONES PROCESADAS

- Débitos y créditos registrados en CTACTES
- Saldos actualizados en tiempo real
- Reportes en TRICKLE-FEED-REPORT/

- Archivo movido a BATCH-DONE/
Tiempo total: ~1-2 minutos para 25k trx
Throughput: 4k-12k transacciones/minuto

Estadísticas:

- **Entrada:** 25,005 registros (136 bytes c/u)
- **Procesamiento:** 6 réplicas en paralelo (~4k trx/min c/u)
- **Salida:** Todos los saldos actualizados, reportes generados
- **Tiempo total:** 1-2 minutos

ESTRUCTURA DE BASE DE DATOS

5.1 Esquema Principal (proyecto_cobol)

5.1.1 Tablas Transaccionales

```
-- CLIENTES: Registro de clientes bancarios
CREATE TABLE CLIENTES (
  ID_CLIENTE          INT AUTO_INCREMENT PRIMARY KEY,
  TIPO_DOC             VARCHAR(10),          -- CC, TI, PP, etc.
  DOC_CLIENTE         VARCHAR(20) UNIQUE,
  NOMBRE              VARCHAR(100),
  APELLIDO            VARCHAR(100),
  EMAIL              VARCHAR(100),
  TELEFONO            VARCHAR(20),
  ESTADO              ENUM('ACTIVO', 'INACTIVO', 'SUSPENDIDO'),
  FECHA_ALTA          TIMESTAMP,
  FECHA_MODIFICACION  TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- CTACTES: Cuentas corrientes (Saldos)
CREATE TABLE CTACTES (
  ID_CTACTE           INT AUTO_INCREMENT PRIMARY KEY,
  ID_CLIENTE          INT NOT NULL,
  NRO_CUENTA          VARCHAR(20) UNIQUE,
  SALDO_ACTUAL         DECIMAL(15,2),        -- En pesos/unidad
  SALDO_DISPONIBLE     DECIMAL(15,2),
  TIPO_CUENTA          ENUM('CORRIENTE', 'AHORRO', 'PLAZO'),
  ESTADO              ENUM('ACTIVA', 'INACTIVA', 'CONGELADA'),
  FECHA_APERTURA      TIMESTAMP,
  FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTES(ID_CLIENTE)
);

-- CREDITOS: Créditos y préstamos
CREATE TABLE CREDITOS (
  NUM_CREDITO         INT AUTO_INCREMENT PRIMARY KEY,
  ID_CLIENTE          INT NOT NULL,
```



```

    MONTO_INICIAL          DECIMAL(15,2),
    SALDO_PENDIENTE        DECIMAL(15,2),
    TASA_INTERES           DECIMAL(5,2),
    CUOTA_MENSUAL          DECIMAL(15,2),
    ESTADO                 ENUM('ACTIVO','CANCELADO','VENCIDO'),
    FECHA_DESEMBOLSO       TIMESTAMP,
    FECHA_VENCIMIENTO      DATE,
    FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTES(ID_CLIENTE)
);

-- TARJETAS: Tarjetas de crédito
CREATE TABLE TARJETAS (
    ID_TARJETA             INT AUTO_INCREMENT PRIMARY KEY,
    ID_CLIENTE             INT NOT NULL,
    NRO_TARJETA            VARCHAR(20) UNIQUE,
    LIMITE_CREDITO         DECIMAL(15,2),
    SALDO_UTILIZADO        DECIMAL(15,2),
    SALDO_DISPONIBLE       DECIMAL(15,2),
    FECHA_VENCIMIENTO      DATE,
    ESTADO                 ENUM('ACTIVA','VENCIDA','CANCELADA','BLOQUEADA'),
    FECHA_EMISION          TIMESTAMP,
    FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTES(ID_CLIENTE)
);

-- HIPOTECAS: Hipotecas (Mortgages)
CREATE TABLE HIPOTECAS (
    ID_HIPOTECA           INT AUTO_INCREMENT PRIMARY KEY,
    ID_CLIENTE            INT NOT NULL,
    MONTO_HIPOTECA        DECIMAL(15,2),
    SALDO_DEUDA           DECIMAL(15,2),
    TASA_INTERES          DECIMAL(5,2),
    CUOTA_MENSUAL          DECIMAL(15,2),
    DIAS_ATRASO           INT DEFAULT 0,
    ESTADO                 ENUM('ACTIVA','PAGADA','MORA','CANCELADA'),
    FECHA_CONSTITUCION     DATE,
    FECHA_VENCIMIENTO      DATE,
    FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTES(ID_CLIENTE)
);

-- INVERSIONES: Inversiones y depósitos a plazo
CREATE TABLE INVERSIONES (
    ID_INVERSION          INT AUTO_INCREMENT PRIMARY KEY,
    ID_CLIENTE            INT NOT NULL,
    MONTO_INICIAL         DECIMAL(15,2),
    SALDO_VIGENTE         DECIMAL(15,2),
    TASA_RENDIMIENTO      DECIMAL(5,2),
    TIPO_INVERSION        VARCHAR(50),          -- Acciones, Bonos, etc.
    ESTADO                 ENUM('ACTIVA','VENCIDA','CANCELADA'),
    FECHA_INICIO           DATE,
    FECHA_VENCIMIENTO      DATE,
    FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTES(ID_CLIENTE)
);

```

5.1.2 Tablas Trickle Feed

```
-- TFFM: Tabla maestra de Trickle Feed
CREATE TABLE TFFM (
  ID_LOTE                INT AUTO_INCREMENT PRIMARY KEY,
  NOMBRE_ARCHIVO         VARCHAR(200),      -- Ej: DEP-999-170526-160000-170526-
001.TXT
  FASE                   INT,               -- 00=Ingresado, 05=Distribuido, 20-
30=Procesando, 40=Finalizado
  TIPO_PROG              VARCHAR(10),      -- EPG, TRF, etc.
  ESTADO_REPLICA         INT,               -- 1-6 (réplica destino)
  TOTAL_REGISTROS        INT,               -- Contador
  REGISTROS_OK           INT DEFAULT 0,
  REGISTROS_ERROR        INT DEFAULT 0,
  FECHA_PROCESO          TIMESTAMP,
  STATUS
ENUM('PENDIENTE','PROCESANDO','EXITOSO','FALLIDO','REINTENTO'),
  TIEMPO_PROCESAMIENTO   INT,               -- En segundos
  OBSERVACIONES          TEXT
);

-- TF01 a TF06: Réplicas de distribución
CREATE TABLE TF01 (
  ID_TF01                INT AUTO_INCREMENT PRIMARY KEY,
  ID_LOTE                INT,               -- FK → TFFM
  ESTADO                 INT,               -- 1=Ingresado, 2=Pendiente,
3=Procesado, 4=Error
  DATOS_TX               VARCHAR(136),      -- Raw 136 bytes posicionales
  CUENTA                 INT,               -- Parsed: ID_CUENTA
  MONTO                  DECIMAL(15,2),     -- Parsed: IMPORTE
  NUM_CREDITO            INT,               -- Parsed: si aplica
  TRACE_ID               VARCHAR(40),      -- Parsed: identificador único
  TERMINAL               VARCHAR(4),        -- Parsed: ATM origin
  TYPE_UPDATE            CHAR(1),          -- D/C/P/R
  (Debit/Credit/Payment/Read)
  COD_ERROR              INT,               -- Si ESTADO=4: 07=Saldo
insuficiente, etc.
  FECHA_PROCESAMIENTO    TIMESTAMP,
  REPLICA_NO             INT,               -- 1 (para TF01)
  INDEX (ID_LOTE),
  INDEX (ESTADO),
  FOREIGN KEY (ID_LOTE) REFERENCES TFFM(ID_LOTE)
);

-- TF02, TF03, TF04, TF05, TF06 (Estructura idéntica)

-- TRICKLE_FEED_REPORT: Reportería
CREATE TABLE TRICKLE_FEED_REPORT (
  ID_REPORTER            INT AUTO_INCREMENT PRIMARY KEY,
  ID_LOTE                INT,
  TOTAL_PROCESADOS       INT,
  PROCESADOS_OK          INT,
  PROCESADOS_ERROR       INT,
```

```

REINTENTADOS          INT,
FECHA_REPORTER        TIMESTAMP,
TIEMPO_PROCESAMIENTO  INT,
FOREIGN KEY (ID_LOTE) REFERENCES TFFM(ID_LOTE)
);

```

5.1.3 Tablas de Auditoría

```

-- AUDIT_MAESTRA: Cierre mensual de clientes
CREATE TABLE AUDIT_MAESTRA (
  ID_AUDIT          INT AUTO_INCREMENT PRIMARY KEY,
  FECHA_PROCESO      DATE,
  TIPO_CONSOLIDACION VARCHAR(50),          -- Ej: CIERRE_MENSUAL_MAYO_2026
  SALDO_INICIAL      DECIMAL(15,2),        -- Suma total antes
  SALDO_FINAL        DECIMAL(15,2),        -- Suma total después
  TRANSACCIONES_PROCESADAS INT,
  CLIENTES_PROCESADOS INT,
  TIMESTAMP          TIMESTAMP
);

-- AUDIT_TARJETAS: Cambios en tarjetas de crédito
CREATE TABLE AUDIT_TARJETAS (
  ID_AUDIT          INT AUTO_INCREMENT PRIMARY KEY,
  FECHA_PROCESO      DATE,
  ID_TARJETA        INT,
  ID_CLIENTE         INT,
  EVENTO             VARCHAR(100),         -- VENCIMIENTO, BLOQUEO, etc.
  SALDO_ANTES        DECIMAL(15,2),
  SALDO_DESPUES      DECIMAL(15,2),
  TIMESTAMP          TIMESTAMP
);

-- AUDIT_HIPOTECAS: Cambios en hipotecas
CREATE TABLE AUDIT_HIPOTECAS (
  ID_AUDIT          INT AUTO_INCREMENT PRIMARY KEY,
  FECHA_PROCESO      DATE,
  ID_HIPOTECA        INT,
  ID_CLIENTE         INT,
  EVENTO             VARCHAR(100),         -- PAGO, MORA, etc.
  DEUDA_ANTES        DECIMAL(15,2),
  DEUDA_DESPUES      DECIMAL(15,2),
  DIAS_MORA          INT,
  TIMESTAMP          TIMESTAMP
);

```

PROCESOS BATCH

6.1 Script: compilar.bat

Propósito: Pre-compilar `.sqb` → `.cob`, compilar a objeto, linkear ejecutables

Procedimiento:

```
@echo off
REM Compilación completa del proyecto CORE TATA

SET COBOL_COMPILER=cobc
SET COBOL_FLAGS=-x -free -Wall
SET PROJECT_ROOT=.
SET SRC_PATH=%PROJECT_ROOT%\src
SET SQL_PATH=%PROJECT_ROOT%\sql
SET OUTPUT_DIR=%PROJECT_ROOT%\bin

ECHO [1/3] Pre-compilando archivos SQL (.sqb a .cob)...
esqlOC.exe /i%SQL_PATH% /o%SRC_PATH%\mainline

ECHO [2/3] Compilando objetos (.cob a .o)...
cd %SRC_PATH%\mainline
%COBOL_COMPILER% -c %COBOL_FLAGS% *.cob

ECHO [3/3] Linkeando ejecutables...
%COBOL_COMPILER% -x %COBOL_FLAGS% -o %OUTPUT_DIR%\BANCSMENU.exe BANCSMENU.o
%COBOL_COMPILER% -x %COBOL_FLAGS% -o %OUTPUT_DIR%\TFDRMAIN.exe TFDRMAIN.o
%COBOL_COMPILER% -x %COBOL_FLAGS% -o %OUTPUT_DIR%\BAT000.exe BAT000.o

ECHO [OK] Compilación finalizada. Ejecutables en bin\
PAUSE
```

6.2 Script: ejecutar.bat

Propósito: Copiar DLLs necesarias, lanzar BANCSMENU.exe

```
@echo off
REM Lanzador de CORE TATA

SET BIN_PATH=%CD%\bin
SET COPY_LIBS=libcob-4.dll libgmp-10.dll libmysqlclient.dll

ECHO [Preparando ambiente...]
FOR %%F IN (%COPY_LIBS%) DO (
    IF NOT EXIST "%BIN_PATH%\%%F" (
        COPY "C:\GnuCOBOL\lib\%%F" "%BIN_PATH%" >NUL
    )
)

ECHO [Iniciando BANCSMENU...]
CD %BIN_PATH%
BANCSMENU.exe
```

```
IF ERRORLEVEL 1 (  
    ECHO [ERROR] Fallo en ejecución  
    PAUSE  
) ELSE (  
    ECHO [OK] BANCSMENU finalizado correctamente  
)
```

6.3 Script: monitor_lotes.bat

Propósito: Monitorear BATCH-UPLOADS-TEMP, registrar en TFFM, gatillar procesamiento

```
@echo off  
REM Monitor de archivos de lotes - Detección automática  
  
SET BATCH_TEMP=.\banco\spool\Interfaces\BATCH-UPLOADS-TEMP\  
SET BATCH_UPLOAD=.\banco\spool\Interfaces\BATCH-UPLOAD-S\  
  
:LOOP  
REM Busca archivos nuevos cada 30 segundos  
ECHO [%DATE% %TIME%] Monitoreando %BATCH_TEMP%...  
  
FOR %%F IN (%BATCH_TEMP%*.txt) DO (  
    ECHO [Encontrado] %%F - Registrando en TFFM...  
  
    REM Inserta en TFFM y registra  
    sqlcl.exe -s root/tata@localhost:3306/proyecto_cobol ^  
        "INSERT INTO TFFM (NOMBRE_ARCHIVO, FASE, TIPO_PROG, STATUS) ^  
        VALUES ('%%F', 00, 'EPG', 'PENDIENTE');"   
  
    REM Mueve a BATCH-UPLOAD-S para procesamiento  
    MOVE "%%F" "%BATCH_UPLOAD%" >NUL  
  
    ECHO [OK] Archivo %%F registrado y movido  
)  
  
TIMEOUT /T 30  
GOTO LOOP
```

GUÍA DE OPERACIÓN

7.1 Inicio del Sistema

Pasos:

1. Compilación inicial:

```
cd c:\Users\rocha\OneDrive\Documentos\PROYECTO CORE TATA\proyecto_cobol  
bin\compilar.bat
```

Salida esperada: [OK] Compilación finalizada. Ejecutables en bin\

2. Lanzar menú principal:

```
bin\ejecutar.bat
```

Se abre menú interactivo en terminal

3. Verificar conexión a BD:

```
Opción: [0] Diagnóstico  
→ testconn.exe  
→ Verifica conexión MySQL
```

7.2 Operación Transaccional (Online)

Opción 1: Registrar nuevo cliente

```
BANCSMENU  
→ Opción: 1 (Gestión de Clientes)  
→ Ingresa: Tipo Doc (CC), Doc (12345678), Nombre (Juan Perez)  
→ DBIOCUSM.cob: INSERT CLIENTES  
→ Resultado: [OK] Cliente registrado ID=1001
```

Opción 2: Realizar débito a cuenta

```
BANCSMENU  
→ Opción: 2 (Cuentas Corrientes)  
→ Ingresa: ID Cliente (1001), Acción (D), Monto (15000 = $150.00)  
→ tkin01.cob: UPDATE CTACTES SALDO-15000  
→ Resultado: [OK] Débito procesado, Saldo: $850.00
```

7.3 Operación Batch: Cierre Mensual

Procedimiento:

BANCSMENU

- Opción: 5 (Cierre Mensual)
- Confirma fecha: 31-05-2026

BAT000.cob inicia:

```
[01/10] Consolidando CLIENTES...
[02/10] Actualizando CTACTES...
[03/10] Procesando TARJETAS...
[04/10] Identificando vencimientos...
[05/10] Procesando HIPOTECAS...
[06/10] Calculando mora...
[07/10] Aplicando pagos automáticos...
[08/10] Insertando AUDIT_MAESTRA...
[09/10] Insertando AUDIT_TARJETAS...
[10/10] Insertan AUDIT_HIPOTECAS...
```

[OK] Cierre COMPLETADO

Próximo proceso: 01-06-2026

7.4 Operación Batch: Trickle Feed Masivo

Pre-requisitos:

1. Generar archivo de entrada (25k transacciones)
2. Colocar en `banco/spool/Interfaces/BATCH-INPUT/`

Generación de datos de prueba:

```
python generar.py
```

```
[Ejecutando generador de 25,005 transacciones...]
```

```
[OK] Archivo masivo generado con éxito
```

```
Ubicación: banco\spool\Interfaces\BATCH-INPUT\DEP-999-170526-160000-170526-001.TXT
```

Procesamiento:

```
bin\monitor_lotes.bat (Opcional: detección automática)
```

- └ Detecta nuevo archivo en BATCH-INPUT
- └ Registra en TFFM
- └ Mueve a BATCH-UPLOADS-TEMP

BANCSMENU

- Opción: 7 (Trickle Feed Batch)
- TFDMAIN.exe inicia:

```
[TFFILE] Ingesta y Validación
```

- └ Leyendo DEP-999-170526-160000-170526-001.TXT...
- └ Parsea 136 bytes posicionales

```
| Valida estructura
| Quiebra en 3 lotes: 10k + 10k + 5k
| Registra ID_LOTE=1234 en TFFM
| Fase: 00 (Ingresado)
└ [OK] TFFILE completado

[TFMX] Distribución a Réplicas
| Distribuyendo a TF01-TF06...
| INSERT TF01 (Lote 1: 10,000 registros)
| INSERT TF02 (Lote 2: 10,000 registros)
| INSERT TF03 (Lote 3: 5,000 registros)
| Fase: 05 (Distribuido)
└ [OK] TFMX completado

[TFBATFIN] Procesador Masivo (PARALELO)
| TF01: Procesando 10,000 transacciones...
| TF02: Procesando 10,000 transacciones...
| TF03: Procesando 5,000 transacciones...
| [PARALELO] Cada réplica procesa ~4k trx/min
| Invoca tkin01.cob x 25,000 veces
| Actualiza CTACTES.SALDO para cada débito/crédito
| Registra COD_RETORNO (00=OK, 07=Error)
| Fase: 20-30 (Procesando)
└ [OK] Todas las réplicas completadas (~1-2 min)

[RRD000] Consolidación y Reportes
| Recolectando resultados...
| Total OK: 24,850
| Total ERROR: 155 (saldo insuficiente, etc.)
| Genera TRICKLE_FEED_REPORT
| Fase: 40 (Finalizado)
| Status: EXITOSO
| Mueve archivo a BATCH-DONE/
└ [OK] RRD000 completado

[FINAL] Procesamiento de 25,005 transacciones en 1m 47s
Throughput: 12,400 transacciones/minuto
```

DEPENDENCIAS TÉCNICAS

8.1 Software Requerido

Componente	Versión	Propósito
GnuCOBOL	4.x (2024.04.30)	Compilador COBOL
esqlOC	v3	Pre-compilador SQL embebido
MySQL Server	8.0+	Base de datos
MySQL Connector ODBC	8.0 ANSI	Driver SQL

Componente	Versión	Propósito
Windows	XP SP3+	Sistema Operativo
Python	3.8+	(Opcional: generador de datos)

8.2 Librerías Requeridas (Runtime)

```

libcob-4.dll      - Runtime COBOL
libgmp-10.dll    - Librería matemática
libmysqlclient.dll - Cliente MySQL ODBC

```

Ubicación: `C:\GnuCOBOL\lib\` (copiadas a `bin\` al ejecutar)

8.3 Estructura de Directorios

```

proyecto_cobol/
├── bin/
│   ├── compilar.bat      # Script compilación
│   ├── ejecutar.bat     # Script ejecución
│   ├── monitor_lotes.bat # Monitor archivo
│   ├── BANCSMENU.exe     # Ejecutable principal
│   ├── TFDMAIN.exe      # Ejecutable TF Batch
│   ├── BAT000.exe       # Ejecutable cierre
│   └── *.dll            # Librerías runtime
├── src/
│   ├── mainline/        # Programas COBOL
│   │   ├── BANCSMENU.cob
│   │   ├── tkin01.cob
│   │   ├── TFFILE.cob
│   │   ├── TFMX.cob
│   │   ├── TFBATFIN.cob
│   │   ├── RRD000.cob
│   │   ├── BAT000.cob
│   │   └── DBIO*.cob (Clientes, Tarjetas, etc.)
│   ├── copies/          # Copybooks compartidos
│   │   ├── LKTF.CPY
│   │   ├── LKCIF.CPY
│   │   ├── PATHS.CPY
│   │   └── *REC.CPY (Record definitions)
│   └── bin/
│       └── ocsql.exp     # Configuración ODBC
└── sql/
    ├── BAT000.sqb       # SQL embebido cierre
    ├── tkin01.sqb       # SQL motor contable
    └── TFFILE.sqb       # SQL ingesta

```

```

├── TFMX.sqb          # SQL distribución
├── TFBATFIN.sqb      # SQL procesamiento
├── DBIOCUSM.sqb      # SQL clientes
├── DBIOTARJ.sqb      # SQL tarjetas
├── DBIOBORM.sqb      # SQL hipotecas
├── DBIOINVM.sqb      # SQL inversiones
├── migrations/       # Scripts inicialización

├── config/
│   ├── environment.properties # Configuración
│   └── paths.cfg             # Rutas

├── banco/
│   └── spool/Interfaces/
│       ├── BATCH-INPUT/      # Entrada cruda
│       ├── BATCH-UPLOAD-S/   # Validados
│       ├── BATCH-UPLOADS-TEMP/ # Tránsito
│       ├── BATCH-DONE/       # Completados
│       └── TRICKLE-FEED-REPORT/# Reportes

├── docs/
│   ├── logs/                # Logs de ejecución
│   └── reporteria/          # Reportes gerenciales

├── test/
│   └── Archivos de prueba

```

8.4 Configuración ODBC

Archivo: src/bin/ocsql.exp

```

USER=root
PASSWORD=tata
HOST=localhost
PORT=3306
DATABASE=proyecto_cobol
DRIVER=MySQL ODBC 8.0 ANSI Driver

```

8.5 Variables de Entorno

```

SET GNUCOBOL_HOME=C:\GnuCOBOL
SET COBOL_LIB=%GNUCOBOL_HOME%\lib
SET COBOL_INCLUDE=%GNUCOBOL_HOME%\include
SET PATH=%PATH%;%COBOL_LIB%
SET DB_HOST=localhost
SET DB_USER=root
SET DB_PASS=tata
SET DB_NAME=proyecto_cobol

```

RESUMEN EJECUTIVO

Características Principales

- ✓ **Sistema integral bancario** con gestión de clientes, cuentas, tarjetas, hipotecas
- ✓ **Operaciones transaccionales online** con respuesta < 100ms
- ✓ **Procesamiento masivo paralelo** (Trickle Feed): 25k transacciones en 1-2 minutos
- ✓ **Cierre mensual automático** con consolidación y auditoría
- ✓ **Arquitectura escalable** con 6 réplicas de procesamiento paralelo
- ✓ **Integridad de datos** con auditoría completa en 3 tablas
- ✓ **Implementado en COBOL** moderno (GnuCOBOL 4.x)

Capacidades Actuales

Métrica	Valor
Transacciones/minuto (TF)	12,400 trx/min (25k en 1.047 min)
Clientes registrados	Ilimitado (escalable con BD)
Cuentas por cliente	1-N
Tarjetas por cliente	1-N
Hipotecas por cliente	1-N
Usuarios simultáneos (Online)	5-10 (limitado por TTY)
Lotes/día (Batch)	~50-100 lotes de 25k transacciones

Próximas Mejoras Sugeridas

1. **Interfaz Web** - Reemplazar TTY con interfaz moderna
2. **Microservicios** - Separar servicios en APIs independientes
3. **Caché en memoria** - Redis para saldos frecuentes
4. **Blockchain** - Auditoría distribuida inmutable
5. **IA/ML** - Detección de fraude en transacciones
6. **Replicación BD** - Alta disponibilidad MySQL

Documento preparado: 19 de Mayo de 2026
Versión: 1.0
Estado: Completo
Clasificación: Documentación Técnica Interna